MIT 8103 Advanced Database Systems - Individual Practical Database Portfolio

Name: Adeoye Marvellous Oluwaferanmi **Matric No:** 2026/A/MIT/0424 **Session:** 2026/2027, First Semester CA

GitHub repository (private, assessor invited): https://github.com/madeoye1746-png/Advanced-Database-Systems

1. Selected organisation / case study

NaijaMart Online - a simulated Nigerian e-commerce platform (realistic business scenario). It sells electronics, fashion, home goods, groceries and beauty products to customers in all regions of Nigeria, delivered by third-party couriers. Users of the database: customers, warehouse staff, finance, logistics and management. Major data: categories, products and stock, customers, orders and order lines, payments, deliveries, stock audit log. Key operations: add products, place an order, deduct stock, take payment, ship and deliver, cancel/refund, sales reports. The same case study is used in all five portfolios (DBMS: PostgreSQL 16 and MongoDB 7 in Docker; 200,000 orders and 500,000 order lines of synthetic data).

2. Portfolio 1 - Database Design and Modelling

- 8 tables in 3NF: category (self-referencing), customer, product, orders, order_item (resolves the M:N between orders and products), payment, delivery, stock_audit.
- Primary/foreign keys, UNIQUE, NOT NULL and CHECK constraints (stock >= 0, valid status lists, delivered_at >= shipped_at), plus a trigger that audits every stock change.
- Normalisation walked from a spreadsheet-style order sheet through 1NF/2NF/3NF; two documented, deliberate denormalisations (price snapshot in order_item, stored order total).
- Evidence: 7 deliberately invalid inserts/updates were all rejected by the database; trigger logged the stock change.

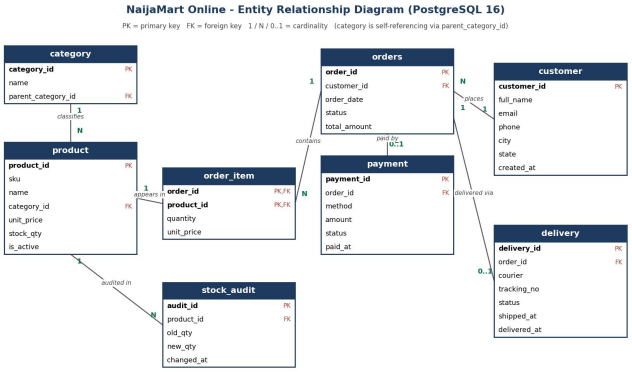


Figure 1: ER diagram (portfolio-1-database-design/er_diagram.png)

3. Portfolio 2 - Query Processing and Optimisation

- 7 business queries (joins, aggregation, sub-queries, NOT IN vs NOT EXISTS, window function) with results; plans captured with EXPLAIN (ANALYZE, BUFFERS) before and after tuning; timings are best of 3 runs.
- Composite index on (customer_id, order_date): customer history 19.6 ms -> 0.18 ms (Seq Scan -> Index Scan, sort removed). Partial index on PENDING orders: 6.6x. Covering index: 3.4x (Index Only Scan).
- Honest finding: an index on order_item was ignored by the planner because the top-products query reads every row; it was removed and a materialized view cut the query from 235 ms to 0.15 ms (trade-off: refresh needed).

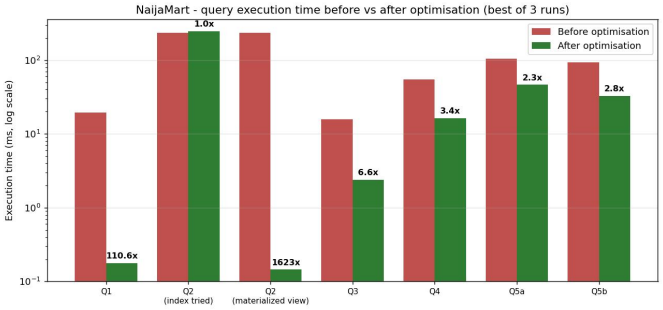


Figure 2: query time before vs after optimisation (log scale)

4. Portfolio 3 - Transactions and Concurrency

- Atomic order placement (order + lines + stock + total): COMMIT on success; an order for more units than in stock fails the CHECK, the whole transaction is rolled back (0 orphan rows, stock unchanged). SAVEPOINT keeps an order while undoing one item. A reusable place_order() function locks the product row with SELECT ... FOR UPDATE.
- Two real parallel sessions with timestamps: LOST UPDATE reproduced under READ COMMITTED (two buyers, stock ended 9 instead of 8) and fixed two ways (atomic UPDATE; SELECT ... FOR UPDATE, where session B visibly waits). Non-repeatable read at READ COMMITTED vs a stable snapshot at REPEATABLE READ; dirty reads impossible in PostgreSQL. DEADLOCK detected by the server (one session aborted) and avoided with a consistent lock order.

5. Portfolio 4 - NoSQL and Advanced Data Models

- Same business modelled in MongoDB using real data exported from PostgreSQL: products (flexible per-category attributes, e.g. phones have ram_gb, shirts have size) and orders (customer snapshot, items, payment and delivery embedded in one document). Embedding vs referencing decisions justified.
- Evidence: CRUD with $inc/$push, nested/array queries, aggregation pipelines ($unwind, $group), $jsonSchema validation (invalid product rejected), key-value style carts with a TTL index. Indexes cut documents examined for 'orders of customer 777' from 30,000 to 2 (explain executionStats). Suitability: good for catalogue and order history; stock/payments stay in PostgreSQL (polyglot persistence). Graph (Neo4j) and column (Cassandra) fits are discussed.

6. Portfolio 5 - Distributed and Cloud Database Exercise

- Built a multi-server Docker deployment: primary + streaming replica (read-only, catches up after restart), reporting query served by the replica. Consistency vs availability measured: asynchronous mode keeps accepting writes with the replica down; synchronous mode returned no response within 5 s (consistency over availability).
- Sharding: orders hash-partitioned by customer_id across two physical servers with postgres_fdw (50,019 / 49,981 rows); single-customer query touches one shard only; cross-shard aggregate equals the unsharded result. Failover: primary stopped, standby promoted (timeline 2), no committed data lost. Cloud mapping (Multi-AZ, read replicas, Atlas, PITR backups, NDPA data residency) discussed; no cloud account was used.

Supporting screenshots for all five portfolios are in the separate file MIT8103_Evidence_Screenshots (uploaded with this document); raw outputs are in the evidence/ folders of the GitHub repository.

7. Individual technical reflection

Design choices. I chose an e-commerce scenario (NaijaMart Online) because one business exercises every topic in the course: stock, orders and payments need strict ACID guarantees, the product catalogue needs flexible attributes, and a growing shop eventually needs replication and sharding. In Portfolio 1 I normalised to 3NF and denormalised only two columns on purpose: order_item.unit_price keeps the price at the time of purchase so later price changes cannot rewrite history, and orders.total_amount avoids re-aggregating 500,000 lines for every report. I did not assume the constraints worked; I wrote negative tests that try to break each rule and kept the database's rejections as evidence. I used PostgreSQL for the transactional core and MongoDB only where documents fit (polyglot persistence), and kept the same case study in all five portfolios.

Challenges and how I resolved them. (1) Optimisation: my first index on order_item was ignored by the planner. EXPLAIN showed why: the top-products query reads every row, so a sequential scan is cheaper than 500,000 index lookups. I removed the useless index and used a materialized view instead (235 ms to 0.15 ms), accepting that it is only as fresh as its last refresh. I timed every query best-of-three after a warm-up run so caching did not distort the before/after comparison. (2) Concurrency: a lost update cannot be shown with one session, so I ran two real parallel sessions with timed pauses. Seeing stock end at 9 instead of 8 made the limits of READ COMMITTED concrete. I fixed it two ways (an atomic UPDATE, and SELECT ... FOR UPDATE), reproduced a deadlock that PostgreSQL resolved by aborting one session, and removed it with a consistent lock order. (3) Distributed systems: synchronous replication blocked writes when the standby was stopped, while asynchronous mode stayed available, so CAP/PACELC became a measured trade-off rather than a definition. A subtle result was that the timed-out write later committed locally, which shows that a client timeout does not mean the write failed and that writes should be idempotent. (4) Data quality: while building the MongoDB attributes I noticed that my generated product names did not match their categories. I corrected the seed script and re-ran every affected output so that all evidence in the repository is consistent.

What I learned. Measure before and after, because an index is a trade-off (storage and slower writes), not a free speed-up. Choose the isolation level and locking strategy from the business rule that must never break, here that stock never goes negative or is oversold. Embed data that is always read together and copy a snapshot where history matters, but keep money and stock in the relational store. The shard key decides which queries stay on one server, which is why I hashed on customer_id.

Limitations and next steps. The data is synthetic, the cluster runs on one machine so network failure is simulated, failover was promoted by hand, and the cloud section is a design discussion rather than a deployment on a cloud account. With more time I would add automatic failover (Patroni), managed sharding (Citus), a graph store for product recommendations, automated regression tests for the SQL scripts, and a deployment on a managed cloud database with point-in-time recovery.

8. AI Use Declaration

- **Tool:** Claude (Anthropic), used through the Claude Code desktop app (model Claude Sonnet 5.5).
- **Used for:** proposing the case study structure, writing the SQL, shell and MongoDB scripts, running them against local Docker databases, generating charts/screenshots from the captured output, and drafting the READMEs and this summary.
- **Verification:** I ran/read through the scripts, checked the outputs against expected behaviour (e.g. constraints reject bad data, lost update gives 9 not 8, deadlock error, replica rejects writes), checked the plans before/after, and I can explain every design choice and result. Screenshots are rendered from real, unedited output files in the repository.

9. References

PostgreSQL Global Development Group (2024) PostgreSQL 16 Documentation: EXPLAIN, Indexes, Transaction Isolation, High Availability/Streaming Replication, Table Partitioning, postgres_fdw. https://www.postgresql.org/docs/16/ | MongoDB Inc. (2024) MongoDB Manual 7.0: Data Modeling, Aggregation, Indexes, TTL Indexes, Schema Validation. https://www.mongodb.com/docs/manual/ | Docker Inc. Docker Hub images postgres:16.0 and mongo:7 | Elmasri, R. and Navathe, S. (2016) Fundamentals of Database Systems, 7th ed., Pearson | Brewer, E. (2012) CAP twelve years later, IEEE Computer 45(2) | Nigeria Data Protection Act (2023). Data used is synthetic and generated by the repository's seed script.